REVIEW 2
DESIGN AND IMPLEMENTATION

Causal-Temporal Machine Learning for Root-Cause Localization in 5G Networks Using Multimodal Observability Data

Reddy SaiBaba (23BDS0019) | Arya Diwakar (23BDS0023) | jeswanth G (23bds0212)

1. Design objective

The system turns multivariate 5G radio-access telemetry into four complementary outputs: a window-level anomaly score, an anomaly-type prediction, KPI-level root-cause scores, and a per-time-step early-warning hazard. The design pairs temporal modeling with estimated KPI dependencies, while keeping preprocessing and evaluation separated by recording and split.

2. Model architecture

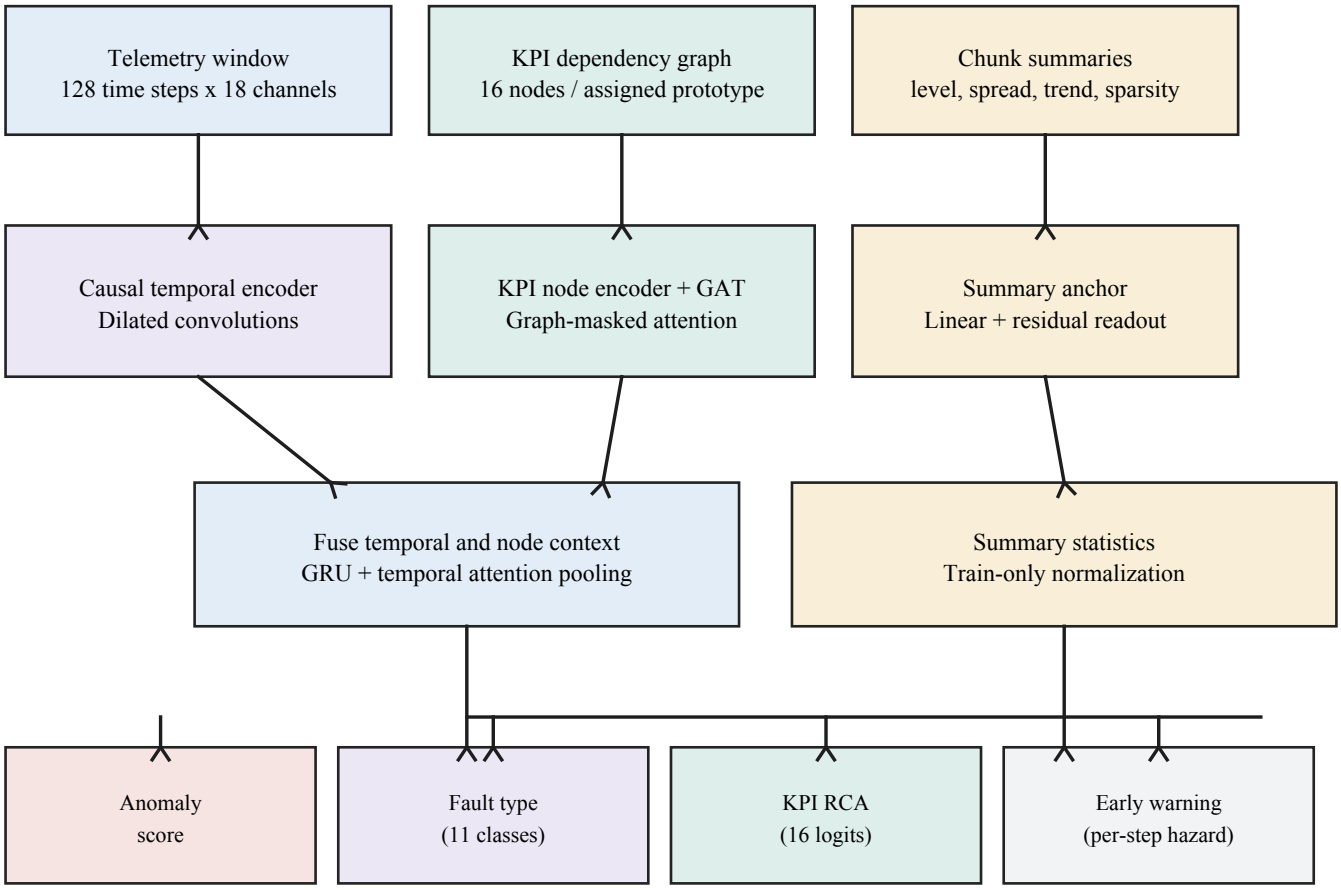


Figure 1. TAGN data flow. The graph is an estimated dependency structure; it is not treated as a verified physical causal DAG.

Design choices

- Temporal path: causal dilated convolutions encode local change without using future samples; a GRU and attention pooling form a window representation.
- Graph path: each KPI has its own temporal node representation; graph attention is masked to estimated edges and retains edge strengths.
- Summary path: an explicit statistical readout keeps level, spread, trend, quantiles and zero fraction available to the anomaly and type heads.

Implementation and evaluation workflow

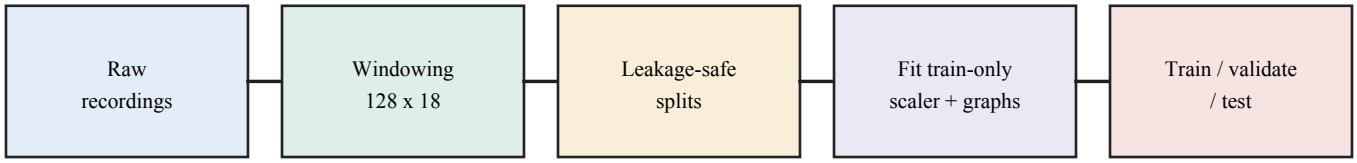


Figure 2. Reproducible implementation pipeline. Splits are recording-aware; fitted preprocessing artifacts are learned from training data only.

3. Data preparation and split safeguards

The corpus is organized as 128-step windows with 16 numeric KPI channels and two protocol-code channels. The chronological split is performed recording by recording; chunks that straddle boundaries are dropped, overlapping anomaly bursts are kept atomic, and source recordings paired with injected synthetic twins are assigned to the same split. Location and scale parameters are fit on training chunks only. The separate leave-one-recording-out protocol refits its scaler and graph prototypes inside each fold.

4. KPI dependency graph

For graph estimation, consecutive chunks are pooled into sliding bins (90 chunks, hop 45). GraphicalLasso estimates a sparse association skeleton from the 16 numeric KPIs; transfer entropy is used to assess edge orientation. Six topology prototypes are learned from training bins, and validation/test bins are assigned to the nearest training prototype. These are exploratory dependencies, not verified physical causes. The distinction is important: the graph constrains attention, but the evaluation does not claim that the estimated edges identify causal mechanisms.

5. Model heads and optimization

The shared encoder feeds anomaly detection, 11-way fault classification, 16-KPI multi-label RCA, and early warning. The anomaly score combines a summary-based linear anchor, a compact summary residual, and the temporal/graph trunk. Type prediction combines trunk and summary context; RCA is produced from time-averaged KPI-node states. The hazard head operates on causal step states and uses a pre-onset mask. Training uses a weighted joint objective (anomaly, type, RCA and masked hazard losses), AdamW, gradient clipping, normal-window undersampling, and validation-based early stopping.

6. Implementation and reproducibility

The implementation is organized into data loading, preprocessing, graph construction, model, experiment, evaluation and reporting modules. Configurations live under configs/; trained weights and run outputs are stored under checkpoints/ and outputs/. Experiments record seeds, configurations and data fingerprints in the results registry. The Streamlit dashboard replays held-out windows and presents telemetry, graph context, predictions and a model what-if panel. A what-if perturbation is a model sensitivity test; it is not an identified real-world intervention.

7. Evaluation protocol

Thresholds are selected on validation data and fixed before test evaluation. Detection is reported with PR-AUC and ROC-AUC, with additional operating-point metrics; RCA uses ranking and multi-label measures; uncertainty is estimated by resampling recordings rather than treating overlapping windows as independent. Paired comparisons score both models on the same resampled recordings. This protocol is designed to limit temporal, recording and pseudo-replication leakage.

Implementation details: graph and learning

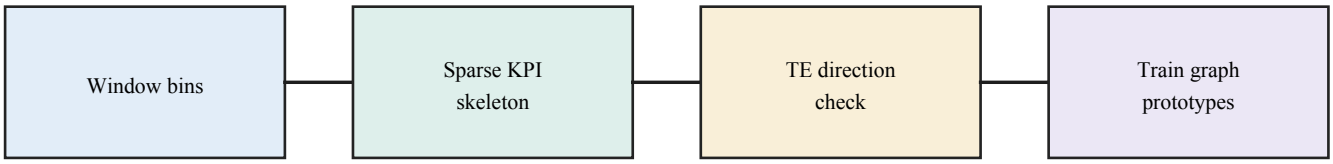


Figure 3. Graph estimation sequence. Prototype fitting uses training bins; held-out bins are assigned to the nearest learned prototype.

8. From telemetry to a graph prototype

The graph builder first groups nearby windows from a recording into overlapping bins. Within each bin, a sparse association skeleton is estimated from the numeric KPI channels. Transfer entropy is evaluated in both directions to assess whether an edge has directional evidence. The resulting bin-level topologies are clustered using training bins only; the assigned prototype supplies the adjacency mask used by the graph-attention layers.

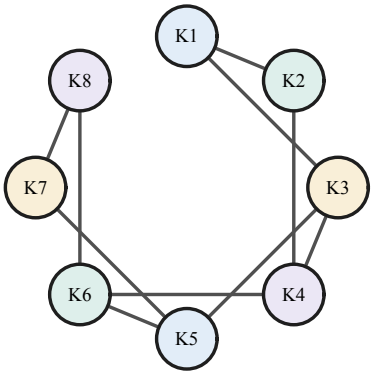


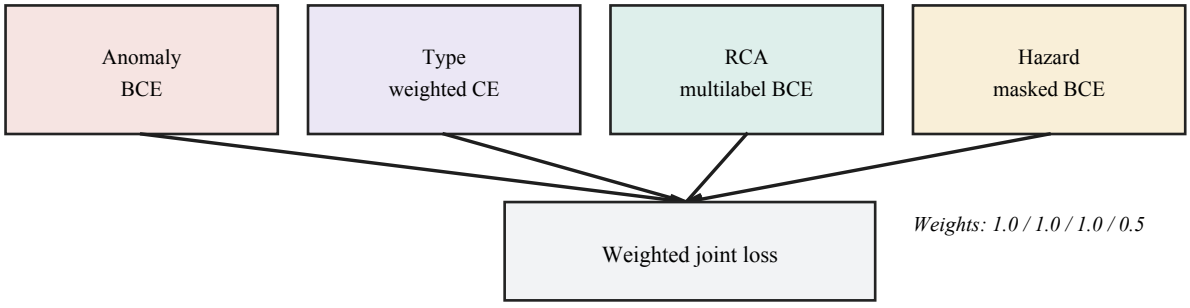
Figure 4. Schematic KPI graph for explanation only; edges shown are illustrative, not measured results.

9. How graph context enters the encoder

Each of the 16 numeric KPIs is encoded as its own temporal node representation. At each time step, attention is computed across KPI nodes and masked to the assigned prototype's edges, with a self-loop for isolated nodes. Edge strength contributes an attention bias. The node states are averaged per time step and fused with the general temporal representation before GRU aggregation. This makes graph structure an inductive bias; it does not make the edge estimates causal facts.

10. Multi-task learning objective

The shared representation supports four tasks. Anomaly detection uses binary cross-entropy; type classification uses class-weighted cross-entropy on anomalous windows; KPI localization uses multi-label binary cross-entropy; and early warning uses a masked hazard loss that only scores the pre-onset region. The configured task weights are 1.0, 1.0, 1.0, and 0.5, respectively.



System operation, code organization, and safeguards

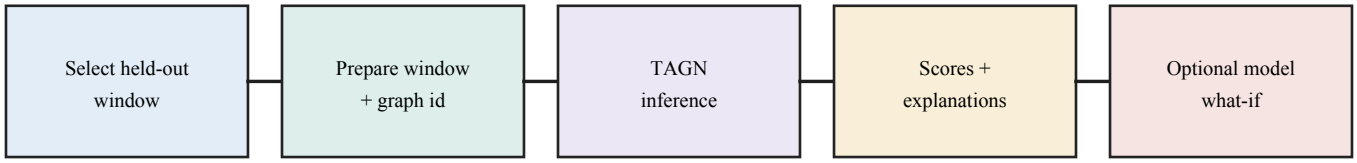


Figure 5. Dashboard inference path. The what-if panel changes model inputs and reports sensitivity; it is not a real network intervention.

11. From trained model to operator-facing view

The Streamlit dashboard can replay a held-out telemetry window and display its KPI traces, graph context, anomaly score, predicted type, ranked KPI causes, and early-warning output. A user can perturb selected KPI values and rerun the checkpoint to inspect score sensitivity. The interface labels this as a model what-if because the observational graph and predictive model do not identify the effect of a real intervention on a live network.

12. Code organization

Data ingestion and adapters are in src/data/; split construction and train-only scaling are in src/preprocessing/; graph estimation and fold-specific graph fitting are in src/models/graphs.py and src/models/graph_fold.py; the core model is src/models/tagn.py. Experiment entry points live in src/experiments/, shared metrics in src/evaluation/, and result aggregation in src/report/. Configuration is kept separately under configs/ so model and training settings can be inspected without reading the experiment code.

13. Leakage and evaluation safeguards

The pipeline enforces several checks at the data boundary: boundary-straddling windows are excluded; overlapping fault bursts remain in one split; synthetic source recordings and injected twins share an assignment; and fitted scalers and graph prototypes never use validation or test recordings. Validation selects operating thresholds and early-stopping checkpoints. Test results are computed afterward, with recording-level bootstrap resampling and paired comparisons to account for clustered windows.

14. Reproduction and verification

A typical reproduction follows the documented sequence: obtain the dataset, build chunks, run leakage-safe preprocessing, generate graph artifacts, train the selected experiment, and regenerate its report. Each run should record its seed, configuration, code revision, and data fingerprint. Unit and regression tests cover preprocessing, model shapes, split integrity, graph checks, and evaluation behavior. The repository's README and experiment modules provide the runnable commands and artifact locations.

15. Design boundaries

The core TAGN and the newer experimental variants are kept conceptually separate. TCN-Transformer backbones, multimodal fusion, intervention utilities, and deferral components should be described as experimental variants unless a corresponding training run and evaluation artifact support the claim. In particular, a graph-based what-if result is evidence about the model's response to a perturbed input, not proof of a physical causal mechanism.